

Technical Documentation

Enterprise Technical Documentation

Version 1.0.0 · 2026-06-21

Hussein A-H

Table of Contents

1. 1. Executive Summary
2. 2. Project Overview
3. 3. Architecture
4. 4. Application Layers
5. 5. Database Design
6. 6. Authentication & Security
7. 7. Domain Events
8. 8. Background Jobs (Celery)
9. 9. Observability
10. 10. API Reference
11. 11. Error Handling
12. 12. Docker & Deployment
13. 13. CI/CD Pipeline
14. 14. Testing Strategy
15. 15. Environment Configuration
16. 16. Feature Walkthroughs
17. 17. Production Hardening
18. 18. Operational Runbooks
19. 19. Future Roadmap
20. 20. Appendix A — Glossary
21. 21. Appendix B — HTTP Status Codes
22. 22. Appendix C — Release History

1. Executive Summary

Expense Tracker API is a production-grade REST backend for personal finance management.

It demonstrates senior-level backend engineering: clean architecture, async I/O, background processing, observability, security hardening, and automated CI/CD.

Release v1.0.0 delivers JWT authentication, transaction management, budgets, categories, audit logging, receipt processing, and async report exports.

The system is designed for containerized deployment behind Nginx with PostgreSQL and Redis as persistent and ephemeral stores respectively.

This document serves engineers, architects, DevOps teams, and technical interviewers evaluating the system's design and operational readiness.

3. System Architecture

Clients reach the API through Nginx in production. FastAPI handles HTTP, authentication, validation, and orchestration. Business rules live in services.

The Unit of Work pattern coordinates repositories and transactions. Domain events decouple side effects (email, audit, Celery) from core request paths.

Celery workers consume tasks from Redis queues: default and email.

OpenTelemetry exports traces from API and workers to Jaeger via OTLP.

4. Application Layers

Presentation: FastAPI routers, Pydantic schemas, dependency injection.

Application: AuthService, TransactionService, BudgetService, etc.

Domain: SQLAlchemy models, specifications for query rules.

Infrastructure: repositories, async sessions, Redis client.

Cross-cutting: middleware, exception handlers, structured logging, telemetry.

5. Database Design

PostgreSQL 16 stores users, categories, transactions, budgets, refresh tokens, audit logs.

Transactions support expense, income, and transfer types with soft delete.

A SQL VIEW `expenses` provides backward compatibility for legacy clients.

Indexes optimize user+date queries, active rows, and GIN tag search.

Budgets enforce unique (user, category, month, year) constraints.

Audit logs are append-only with JSONB metadata for flexible forensics.

Table	Purpose	Key constraints
users	Identity	Unique email, UUID PK
transactions	Financial records	amount > 0, type enum
categories	Classification	Unique per user
budgets	Spending limits	alert_threshold 0–1
refresh_tokens	Session rotation	Hashed token, expiry
audit_logs	Compliance trail	Append-only

6. Authentication & Security

Passwords hashed with bcrypt. Access JWT (HS256) expires in 15 minutes.

Refresh tokens stored hashed in DB; rotation on each refresh call.

Logout blacklists access token JTI in Redis until natural expiry.

OAuth2 /auth/token accepts form username/password for Swagger UI.

JSON /auth/login preserved for API clients using email field.

Password reset uses Redis-stored single-use tokens + email task.

Admin routes guarded by require_admin dependency.

7. Domain Events

Event	Trigger	Handlers
UserRegistered	Registration	Welcome email, audit
TransactionCreated	POST transaction	Audit
TransactionUpdated	PATCH transaction	Audit
TransactionDeleted	DELETE	Audit
BudgetThresholdExceeded	Celery evaluation	Alert email, audit

8. Background Jobs

Celery Beat schedules hourly budget checks, daily token cleanup, and monthly audit maintenance.

Tasks use `task_acks_late` and `reject_on_worker_lost` for reliability.

Email tasks routed to dedicated queue for isolation.

Task	Queue	Trigger	Outcome
send_welcome_email	email	UserRegistered event	On registration
send_budget_alert_email	email	BudgetThresholdExceeded	Budget alert
send_password_reset_email	email	Password reset request	Auth flow
evaluate_all_budgets	default	Celery Beat hourly	Threshold scan
export_transactions_csv	default	HTTP report request	CSV to exports/
generate_monthly_report	default	HTTP or Beat	Monthly summary
process_receipt_image	default	Receipt upload	Pillow processing
cleanup_expired_tokens	default	Beat daily	Refresh token hygiene
archive_audit_partitions	default	Beat monthly	Audit maintenance

9. Observability

OpenTelemetry instruments FastAPI, SQLAlchemy, Redis, and Celery.

Jaeger UI for distributed trace analysis.

Prometheus scrapes metrics; Grafana for dashboards.

Structured logging via structlog with request correlation.

Health endpoints: `/live` (process up), `/ready` (DB + Redis).

10. API Reference

Method	Path	Description	Auth
POST	<code>/api/v1/auth/register</code>	Register a new user account	Public
POST	<code>/api/v1/auth/login</code>	JSON login (email + password)	Public

POST	/api/v1/auth/token	OAuth2 password grant (Swagger)	Public
POST	/api/v1/auth/refresh	Rotate refresh token	Authenticated
POST	/api/v1/auth/logout	Revoke tokens and blacklist JWT	Authenticated
POST	/api/v1/auth/password-reset/request	Request password reset email	Public
POST	/api/v1/auth/password-reset/confirm	Confirm reset with token	Public
GET	/api/v1/users/me	Current user profile	Authenticated
PATCH	/api/v1/users/me	Update profile	Authenticated
PUT	/api/v1/users/me/password	Change password	Authenticated
DELETE	/api/v1/users/me	Deactivate account	Authenticated
GET	/api/v1/users	List users (admin)	Admin
POST	/api/v1/transactions	Create transaction	Authenticated
GET	/api/v1/transactions	List with filters/pagination	Authenticated
GET	/api/v1/transactions/{id}	Get transaction	Authenticated
PATCH	/api/v1/transactions/{id}	Update transaction	Authenticated
DELETE	/api/v1/transactions/{id}	Soft delete	Authenticated
POST	/api/v1/transactions/{id}/receipts	Upload receipt image	Authenticated
GET	/api/v1/categories	List categories	Authenticated
POST	/api/v1/categories	Create category	Authenticated
PATCH	/api/v1/categories/{id}	Update category	Authenticated
DELETE	/api/v1/categories/{id}	Delete category	Authenticated
POST	/api/v1/budgets	Create budget	Authenticated
GET	/api/v1/budgets	List budgets	Authenticated

GET	/api/v1/budgets/summary	Budget vs spend summary	Authenticated
PATCH	/api/v1/budgets/{id}	Update budget	Authenticated
DELETE	/api/v1/budgets/{id}	Delete budget	Authenticated
POST	/api/v1/reports/export/csv	Queue CSV export task	Authenticated
POST	/api/v1/reports/export/monthly	Queue monthly report	Authenticated
GET	/api/v1/reports/tasks/{task_id}	Poll Celery task status	Authenticated
GET	/api/v1/audit/logs	Query audit trail	Admin
GET	/api/v1/health/live	Liveness probe	Public
GET	/api/v1/health/ready	Readiness (DB + Redis)	Public

10.1 POST /api/v1/auth/register

Endpoint 1: POST /api/v1/auth/register

Description: Register a new user account

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.2 POST /api/v1/auth/login

Endpoint 2: POST /api/v1/auth/login

Description: JSON login (email + password)

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.3 POST /api/v1/auth/token

Endpoint 3: POST /api/v1/auth/token

Description: OAuth2 password grant (Swagger)

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.4 POST /api/v1/auth/refresh

Endpoint 4: POST /api/v1/auth/refresh

Description: Rotate refresh token

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.5 POST /api/v1/auth/logout

Endpoint 5: POST /api/v1/auth/logout

Description: Revoke tokens and blacklist JWT

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.6 POST /api/v1/auth/password-reset/request

Endpoint 6: POST /api/v1/auth/password-reset/request

Description: Request password reset email

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.7 POST /api/v1/auth/password-reset/confirm

Endpoint 7: POST /api/v1/auth/password-reset/confirm

Description: Confirm reset with token

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.8 GET /api/v1/users/me

Endpoint 8: GET /api/v1/users/me

Description: Current user profile

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.9 PATCH /api/v1/users/me

Endpoint 9: PATCH /api/v1/users/me

Description: Update profile

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.10 PUT /api/v1/users/me/password

Endpoint 10: PUT /api/v1/users/me/password

Description: Change password

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.11 DELETE /api/v1/users/me

Endpoint 11: DELETE /api/v1/users/me

Description: Deactivate account

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.12 GET /api/v1/users

Endpoint 12: GET /api/v1/users

Description: List users (admin)

Authorization: Admin

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.13 POST /api/v1/transactions

Endpoint 13: POST /api/v1/transactions

Description: Create transaction

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.14 GET /api/v1/transactions

Endpoint 14: GET /api/v1/transactions

Description: List with filters/pagination

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.15 GET /api/v1/transactions/{id}

Endpoint 15: GET /api/v1/transactions/{id}

Description: Get transaction

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.16 PATCH /api/v1/transactions/{id}

Endpoint 16: PATCH /api/v1/transactions/{id}

Description: Update transaction

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.17 DELETE /api/v1/transactions/{id}

Endpoint 17: DELETE /api/v1/transactions/{id}

Description: Soft delete

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.18 POST /api/v1/transactions/{id}/receipt

Endpoint 18: POST /api/v1/transactions/{id}/receipt

Description: Upload receipt image

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.19 GET /api/v1/categories

Endpoint 19: GET /api/v1/categories

Description: List categories

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.20 POST /api/v1/categories

Endpoint 20: POST /api/v1/categories

Description: Create category

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.21 PATCH /api/v1/categories/{id}

Endpoint 21: PATCH /api/v1/categories/{id}

Description: Update category

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.22 DELETE /api/v1/categories/{id}

Endpoint 22: DELETE /api/v1/categories/{id}

Description: Delete category

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.23 POST /api/v1/budgets

Endpoint 23: POST /api/v1/budgets

Description: Create budget

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.24 GET /api/v1/budgets

Endpoint 24: GET /api/v1/budgets

Description: List budgets

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.25 GET /api/v1/budgets/summary

Endpoint 25: GET /api/v1/budgets/summary

Description: Budget vs spend summary

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.26 PATCH /api/v1/budgets/{id}

Endpoint 26: PATCH /api/v1/budgets/{id}

Description: Update budget

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.27 DELETE /api/v1/budgets/{id}

Endpoint 27: DELETE /api/v1/budgets/{id}

Description: Delete budget

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.28 POST /api/v1/reports/export/csv

Endpoint 28: POST /api/v1/reports/export/csv

Description: Queue CSV export task

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.29 POST /api/v1/reports/export/monthly

Endpoint 29: POST /api/v1/reports/export/monthly

Description: Queue monthly report

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.30 GET /api/v1/reports/tasks/{task_id}

Endpoint 30: GET /api/v1/reports/tasks/{task_id}

Description: Poll Celery task status

Authorization: Authenticated

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.31 GET /api/v1/audit/logs

Endpoint 31: GET /api/v1/audit/logs

Description: Query audit trail

Authorization: Admin

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.32 GET /api/v1/health/live

Endpoint 32: GET /api/v1/health/live

Description: Liveness probe

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

10.33 GET /api/v1/health/ready

Endpoint 33: GET /api/v1/health/ready

Description: Readiness (DB + Redis)

Authorization: Public

Request and response bodies defined in OpenAPI schema at /docs.

Errors return consistent JSON: error.code, error.message, error.details.

11. Error Handling

Centralized exception handlers map domain errors to HTTP status.

AuthenticationError → 401, AuthorizationError → 403, ValidationError → 422.
NotFoundError → 404, ConflictError → 409, BusinessRuleError → 400.
Unexpected exceptions logged and returned as 500 without leaking internals.

12. Docker & Deployment

Development: docker-compose.yml with bind mounts and hot reload.
Production: docker-compose.prod.yml overlay with multi-stage images.
API image: non-root user, 4 uvicorn workers, healthcheck.
Celery prod image: dedicated worker CMD (not uvicorn).
Shared volumes for uploads/exports between API and workers.
Nginx prod: rate limiting, security headers, 10MB body limit.

13. CI/CD Pipeline

ci.yml: lint (black, isort, flake8) + integration tests on PR.
Services: Postgres 16, Redis 7 in GitHub Actions.
docker-image job builds and pushes to GHCR on main.
cd.yml: manual or tag-triggered deploy bundle to GHCR.

14. Testing Strategy

47 integration tests cover auth, OAuth2, CRUD domains, Celery wiring.
conftest.py provides transactional rollback per test.
Fixtures: test user, admin, auth headers, mock SMTP.
CI runs full integration suite after alembic upgrade head.
Future: unit tests for services and specifications.

15. Environment Configuration

Variable	Purpose	Notes
SECRET_KEY	JWT signing key (32+ chars in prod)	Required
DB_HOST / DB_USER / DB_PASSWORD / DB_NAME	PostgreSQL primary	Required

REDIS_HOST / REDIS_PASSWORD	Cache, Celery, blacklist	Required
CELERY_BROKER_URL	Redis broker URL	Required
CORS_ORIGINS	Comma-separated origins	Required in staging/prod
OPENAPI_ENABLED	Expose /docs	false in production
OTEL_EXPORTER_ENDPOINT	Jaeger OTLP	http://jaeger:4317
SMTP_HOST / SMTP_USER / SMTP_PASSWORD	Email delivery	Optional dev
ENVIRONMENT	development staging production testing	development

16. Walkthrough: User registration and welcome email

1. Client POSTs /api/v1/auth/register with email, password, profile fields.
2. AuthService validates password policy, hashes with bcrypt, persists user.
3. UserRegistered domain event fires; handler enqueues send_welcome_email.
4. Celery worker sends HTML email via SMTP asynchronously.
5. Audit log records USER_REGISTERED with actor metadata.

16. Walkthrough: Transaction with receipt processing

1. Authenticated POST /api/v1/transactions creates expense row.
2. Optional POST .../receipt uploads multipart image to uploads/.
3. receipt_dispatch enqueues process_receipt_image Celery task.
4. Worker validates image, may resize/thumbnail via Pillow.
5. TransactionUpdated event and audit entry on completion.

16. Walkthrough: Budget threshold alert

1. User defines budget per category/month via POST /api/v1/budgets.
2. Hourly Beat task evaluate_all_budgets aggregates spend.
3. When spend/limit exceeds alert_threshold, BudgetThresholdExceeded fires.
4. send_budget_alert_email notifies user; alert_sent flag prevents spam.

16. Walkthrough: Report export workflow

1. POST `/api/v1/reports/export/csv` returns `task_id` immediately.
2. Client polls GET `/api/v1/reports/tasks/{task_id}` for status.
3. Worker queries transactions, writes CSV to shared exports volume.
4. Result includes download path or presigned-style URL pattern.

16. Walkthrough: OAuth2 Swagger authorization

1. Swagger Authorize uses OAuth2PasswordBearer scheme.
2. Form POST `/api/v1/auth/token` with `username=email`, `password`.
3. Returns `access_token` + `refresh_token` (`token_type` bearer).
4. Swagger injects Authorization header on protected endpoints.
5. GET `/api/v1/users/me` validates JWT via `get_current_user` dep.

17. Production Hardening

SECRET_KEY minimum 32 characters enforced in staging/production.

CORS_ORIGINS must be explicit — no wildcard in prod.

TRUSTED_HOSTS restricts Host header attacks.

OPENAPI_ENABLED=false hides /docs in production.

Run as non-root container user. Rotate all example secrets.

See docs/security/SECURITY.md for full matrix.

18. Operational Runbooks

Deploy staging: `bash scripts/deploy-staging.sh .env.staging`

Migrate: `docker exec api alembic upgrade head`

Logs: `docker logs -f api celery-worker`

Scale workers: increase celery-worker replicas in compose.

Rollback: redeploy previous GHCR tag, run downgrade if needed.

Monitor: Jaeger traces, Grafana dashboards, Flower task queue.

19. Future Roadmap

API v2 when breaking changes are planned.

Kubernetes Helm chart for cloud-native deploy.

Rate limiting middleware beyond Nginx.

OpenAPI client SDK generation.

Expanded unit test coverage.

Real CD smoke tests against staging URL.

20. Appendix A — Glossary

Term	Definition
UoW	Unit of Work — transaction boundary for repositories
JTI	JWT ID — used for Redis blacklist
OTLP	OpenTelemetry Protocol
GHCR	GitHub Container Registry
Beat	Celery periodic task scheduler

21. Appendix B — HTTP Status Codes

Code	Usage
200	Success
201	Created
204	No content (logout, delete)
401	Not authenticated
403	Forbidden
404	Not found
409	Conflict
422	Validation error

500	Internal error
-----	----------------

22. Appendix C — Release History

v1.0.0-rc.1 — Release candidate with CI/CD, prod overlay, Celery jobs.

v1.0.0 — Public GA: OAuth2 fix, documentation, portfolio assets.

10.x Detailed Reference — Authentication

This section documents the Authentication module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in error.details.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

POST /api/v1/auth/register — Register a new user account. Authorization: Public.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

POST /api/v1/auth/login — JSON login (email + password). Authorization: Public.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

POST /api/v1/auth/token — OAuth2 password grant (Swagger). Authorization: Public.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

POST /api/v1/auth/refresh — Rotate refresh token. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/auth/logout` — Revoke tokens and blacklist JWT. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/auth/password-reset/request` — Request password reset email. Authorization: Public.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/auth/password-reset/confirm` — Confirm reset with token. Authorization: Public.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

10.x Detailed Reference — Users

This section documents the Users module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in `error.details`.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

GET `/api/v1/users/me` — Current user profile. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

PATCH `/api/v1/users/me` — Update profile. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

PUT `/api/v1/users/me/password` — Change password. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

DELETE `/api/v1/users/me` — Deactivate account. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

GET `/api/v1/users` — List users (admin). Authorization: Admin.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

10.x Detailed Reference — Transactions

This section documents the Transactions module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in `error.details`.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

POST `/api/v1/transactions` — Create transaction. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

GET `/api/v1/transactions` — List with filters/pagination. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

GET `/api/v1/transactions/{id}` — Get transaction. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

PATCH `/api/v1/transactions/{id}` — Update transaction. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

DELETE `/api/v1/transactions/{id}` — Soft delete. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/transactions/{id}/receipt` — Upload receipt image. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

10.x Detailed Reference — Categories & Budgets

This section documents the Categories & Budgets module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in `error.details`.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

GET `/api/v1/categories` — List categories. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/categories` — Create category. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

PATCH `/api/v1/categories/{id}` — Update category. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

DELETE `/api/v1/categories/{id}` — Delete category. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

POST `/api/v1/budgets` — Create budget. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

GET `/api/v1/budgets` — List budgets. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

GET `/api/v1/budgets/summary` — Budget vs spend summary. Authorization: Authenticated.

Example request uses Content-Type `application/json` unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses `page/per_page` query params where applicable (default `page=1`, `per_page=20`).

PATCH /api/v1/budgets/{id} — Update budget. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

10.x Detailed Reference — Reports & Audit

This section documents the Reports & Audit module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in error.details.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

DELETE /api/v1/budgets/{id} — Delete budget. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

POST /api/v1/reports/export/csv — Queue CSV export task. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

POST /api/v1/reports/export/monthly — Queue monthly report. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

GET /api/v1/reports/tasks/{task_id} — Poll Celery task status. Authorization: Authenticated.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

GET /api/v1/audit/logs — Query audit trail. Authorization: Admin.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

10.x Detailed Reference — Health

This section documents the Health module endpoints in detail.

All authenticated routes require a valid Bearer access token unless marked Public.

Validation errors return HTTP 422 with field-level details in error.details.

Rate limiting is enforced at the Nginx layer in production (20 req/s default).

GET /api/v1/health/live — Liveness probe. Authorization: Public.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

GET /api/v1/health/ready — Readiness (DB + Redis). Authorization: Public.

Example request uses Content-Type application/json unless OAuth2 form endpoint.

Successful responses follow Pydantic response models documented in OpenAPI.

Idempotency: POST create endpoints are not idempotent; clients should retry safely.

Pagination uses page/per_page query params where applicable (default page=1, per_page=20).

23. Appendix D — Engineering Deep Dive 1: Separation of concerns between routes and services

Topic 1: Separation of concerns between routes and services.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 2: Repository pattern and testability

Topic 2: Repository pattern and testability.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 3: Async session lifecycle in request scope

Topic 3: Async session lifecycle in request scope.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 4: Domain events vs direct Celery calls

Topic 4: Domain events vs direct Celery calls.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 5: Refresh token rotation threat model

Topic 5: Refresh token rotation threat model.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 6: Budget alert deduplication via `alert_sent`

Topic 6: Budget alert deduplication via `alert_sent`.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 7: Soft delete semantics for transactions

Topic 7: Soft delete semantics for transactions.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 8: Audit log immutability guarantees

Topic 8: Audit log immutability guarantees.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 9: Read replica routing for list endpoints

Topic 9: Read replica routing for list endpoints.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 10: Nginx as edge security control point

Topic 10: Nginx as edge security control point.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 11: Container health checks and orchestrator integration

Topic 11: Container health checks and orchestrator integration.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 12: Structured logging for incident response

Topic 12: Structured logging for incident response.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 13: Migration strategy with Alembic

Topic 13: Migration strategy with Alembic.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 14: Seed data for categories script

Topic 14: Seed data for categories script.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 15: Error code taxonomy for API clients

Topic 15: Error code taxonomy for API clients.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 16: File upload size limits and validation

Topic 16: File upload size limits and validation.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 17: CSV export memory considerations

Topic 17: CSV export memory considerations.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 18: Receipt image processing pipeline

Topic 18: Receipt image processing pipeline.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 19: Password policy and bcrypt cost factor

Topic 19: Password policy and bcrypt cost factor.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 20: Admin RBAC and least privilege

Topic 20: Admin RBAC and least privilege.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 21: CORS preflight handling

Topic 21: CORS preflight handling.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 22: TrustedHost header validation

Topic 22: TrustedHost header validation.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 23: OpenAPI disable in production

Topic 23: OpenAPI disable in production.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 24: Redis key naming conventions

Topic 24: Redis key naming conventions.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 25: Celery task retry and `acks_late`

Topic 25: Celery task retry and `acks_late`.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 26: Beat schedule timezone UTC

Topic 26: Beat schedule timezone UTC.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 27: Flower exposure in dev only

Topic 27: Flower exposure in dev only.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 28: Prometheus scrape intervals

Topic 28: Prometheus scrape intervals.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 29: Grafana dashboard provisioning

Topic 29: Grafana dashboard provisioning.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 30: Staging vs production env parity

Topic 30: Staging vs production env parity.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 31: Secret rotation runbook

Topic 31: Secret rotation runbook.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 32: Backup and restore for PostgreSQL

Topic 32: Backup and restore for PostgreSQL.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 33: Horizontal scaling of API replicas

Topic 33: Horizontal scaling of API replicas.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 34: Worker queue isolation for email

Topic 34: Worker queue isolation for email.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 35: Load testing with locust (dev dep)

Topic 35: Load testing with locust (dev dep).

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 36: Integration test isolation strategy

Topic 36: Integration test isolation strategy.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 37: Fixture design in conftest.py

Topic 37: Fixture design in conftest.py.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 38: CI service container networking

Topic 38: CI service container networking.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 39: GHCR image tagging conventions

Topic 39: GHCR image tagging conventions.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

23. Appendix D — Engineering Deep Dive 40: Deploy bundle contents in CD workflow

Topic 40: Deploy bundle contents in CD workflow.

The Expense Tracker API prioritizes maintainability through strict layer boundaries.

Services never import FastAPI types; dependencies flow inward only.

Repositories encapsulate SQL; specifications encapsulate filter rules.

This separation enables testing business logic independently and swapping infrastructure without route changes.

Async SQLAlchemy ensures non-blocking I/O under concurrent load.

Celery offloads slow paths: email, reports, image processing.

Event handlers remain idempotent where possible to survive retries.

Production deployments should validate readiness before receiving traffic.

Observability data supports post-incident analysis and latency tuning.

Each module is documented in OpenAPI with request/response schemas.

Clients should implement exponential backoff on 503 readiness failures.

Database migrations must be applied before rolling out new API versions.

24. Appendix E — Operations Checklist 1

Operational checklist item set 1 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 2

Operational checklist item set 2 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 3

Operational checklist item set 3 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 4

Operational checklist item set 4 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 5

Operational checklist item set 5 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 6

Operational checklist item set 6 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 7

Operational checklist item set 7 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 8

Operational checklist item set 8 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 9

Operational checklist item set 9 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 10

Operational checklist item set 10 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.

24. Appendix E — Operations Checklist 11

Operational checklist item set 11 for production readiness verification.

Verify SECRET_KEY length and uniqueness across all environments.

Confirm CORS_ORIGINS matches deployed frontend URLs exactly.

Validate SMTP credentials and send test welcome email.

Run alembic current and compare with expected head revision.

Execute integration test suite in staging before tag promotion.

Review Jaeger traces for error spans during smoke test.

Confirm Celery worker consumes both default and email queues.

Validate Nginx rate limit and body size for receipt uploads.

Document rollback image tag in runbook before production deploy.

Schedule secret rotation calendar for quarterly review.

Archive audit logs per retention policy after partition task.